

The §6.3 "Blow-up to 10^{30} ": Analysis of the Suspicious Point in the v1 Essay

Companion analysis to `geometric_integration_lienard_essay_v1.md` (the **v1** essay), responding to the single open issue raised in the review `geometric_integration_lienard_essay_v1_r1_Cla.md`. The v1 essay is **not** modified here; this document is a self-contained analysis of one specific claim, with all numbers produced by the accompanying scripts (run under the project venv).

0. Verdict in one paragraph

The review's suspicion is **confirmed, on two independent levels**.

1. The "blow-up to $\sim 10^{29}$ - 10^{32} " is an artifact of an implementation bug, not a property of the implicit midpoint method. The Newton solver used in `geometric_integration_computation3.py / 4.py` (the scripts that produced the §6.3 table) has a **sign error in the (2,1) entry of its Jacobian**. At the *first fast jump* of the $\mu = 100$ cycle ($t \approx 81$), the broken Newton iteration runs to its 100-iteration cap without converging (final residual $\sim 10^{81}$ - 10^{90}), and the script **silently returns the last (garbage) iterate** — a point $\sim 10^{27}$ - 10^{29} away from the true state. The entire "blow-up" is produced in a **single failed Newton solve**; the rest of the run merely carries the garbage along.
2. **Even the raw (buggy) values are misquoted in the essay's table**. The raw output `geometric_integration_results4.txt` contains 1.03×10^{27} , 1.44×10^{29} , 8.73×10^{29} for the three implicit-midpoint rows, but the essay's table prints " $\sim 1.0 \times 10^{30}$ ", " $\sim 1.4 \times 10^{32}$ ", " $\sim 8.7 \times 10^{29}$ " — two of the three exponents are off by a factor 10^3 (and the Strang $h = 0.2$ row is off by a factor 10^{-3}). The headline " $\sim 10^{30}$ " propagates the corrupted table, not the raw output.

With a **correct** Newton solve (exact Jacobian, cross-checked against an independent `scipy.optimize.root` solve to machine precision), the implicit midpoint rule on the $\mu = 100$ van der Pol cycle is **bounded at all three step sizes** — at $h = 0.01$: $\max |x| = 2.09$, $\max |v| = 144$ over 10 periods — matching the review's independent reproduction ("values up to ~ 100 - 150 "). The essay's *qualitative* thesis (symplectic + A -stable $\neq L$ -stable, so the stiff jump is mishandled; Radau IIA is the tool that works) **survives, but in a weaker and different form**: the correct method does not escape at all — it produces a *bounded but distorted* cycle (period $T = 179.08$ vs. true 162.84 , a +10% error at $h = 0.01$; $T = 286.5$, +76%, at $h = 0.1$). The quantitative punchline of §6.3 must be withdrawn and replaced by this.

1. The suspicious point, as stated

The essay (**v1, §6.3**) integrates the $\mu = 100$ van der Pol cycle from $(x, v) = (2, 0)$ for three periods with the implicit midpoint rule and reports:

method (step h)	$\max x $ over run	status (as printed)
implicit midpoint, $h = 0.5$	$\sim 1.0 \times 10^{30}$	blow-up

implicit midpoint, $h = 0.1$	$\sim 1.4 \times 10^{32}$	blow-up
implicit midpoint, $h = 0.01$	$\sim 8.7 \times 10^{29}$	blow-up

and states that the method “escapes to $\sim 10^{30}$ even at $h = 0.01$ ”, attributing this to the method being “ A -stable but not L -stable” ($R(-\infty) = -1 \neq 0$), with the residual fast component “re-amplified by the unstable central region $|x| < 1$ ”.

The review (r1_Cla) re-implemented the implicit midpoint rule independently (Newton with the exact Jacobian, tight tolerance) and found that the orbit “does **not** blow up to 10^{29} - 10^{32} ; it stays bounded and even partially recovers, over runs as long as 20 periods”, with “values up to ~ 100 - 150 against a true amplitude of ~ 2 ”. The review suspected “a solver bug in their own script (e.g. a fixed-point iteration that fails to converge rather than a true Newton solve, which can produce spurious garbage that looks like blow-up)”.

The question this analysis settles: **where do the 10^{29} - 10^{32} numbers come from, and what does the implicit midpoint rule actually do on this problem?**

2. Where the numbers come from: the scripts and the raw output

The §6.3 table was produced by `geometric_integration_computation4.py`, whose raw output is `geometric_integration_results4.txt`. Comparing the two, row by row:

row	raw output ('re-sults4.txt')	essay table (§6.3)	verdict
$IM, h = 0.5$	$\max x = 1.0297 \times 10^{27}$	$\sim 1.0 \times 10^{30}$	misquoted ($\times 10^3$)
$IM, h = 0.1$	$\max x = 1.4397 \times 10^{29}$	$\sim 1.4 \times 10^{32}$	misquoted ($\times 10^3$)
$IM, h = 0.01$	$\max x = 8.7312 \times 10^{29}$	$\sim 8.7 \times 10^{29}$	quoted correctly
Strang, $h = 0.5$	$\max x = 2.0062 \times 10^{15}$	$\sim 2.0 \times 10^{15}$	quoted correctly
Strang, $h = 0.2$	$\max x = 4.5917 \times 10^8$	$\sim 4.6 \times 10^5$	misquoted ($\div 10^3$)

Radau IIA, $h = 0.5$ / 1.0	$T = 162.8371$	162.8371	quoted correctly
---------------------------------------	----------------	----------	------------------

So the essay's table is not even faithful to its own raw output: the two "blow-up" magnitudes that feed the " $\sim 10^{30}$ " headline are each a factor 10^3 above what the script actually printed. (This does not by itself answer the review's question — the raw values are still enormous and still not reproduced by the reviewer's implementation — but it is part of "the values could be much lower than claimed": the *printed* values are 10^3 higher than the *computed* ones.)

The rows that do not involve the Newton solver (Strang split, Radau IIA) reproduce **exactly** under re-run (§5 below), which isolates the discrepancy to the implicit-midpoint Newton code path.

3. The Newton bug

3.1 The stage equations and their exact Jacobian

The implicit midpoint step from (x, v) with step h solves for the *endpoint* (x_g, v_g) via the midpoint $(m_x, m_v) = \frac{1}{2}(x + x_g, v + v_g)$:

$$x_g = x + h m_v, \quad v_g = v + h(\mu(1 - m_x^2) m_v - m_x), \quad m_x = \frac{x+x_g}{2}, \quad m_v = \frac{v+v_g}{2}. \quad (\text{N1})$$

Writing $G(x_g, v_g) = (x_g - x - h m_v, v_g - v - h(\mu(1 - m_x^2) m_v - m_x))$, the exact Newton Jacobian (derived symbolically with SymPy; script output (1) of `suspicion63_analysis.py`) is

$$DG = \begin{pmatrix} 1 & -h/2 \\ -\frac{h}{2} J_{21} & 1 - \frac{h}{2} J_{22} \end{pmatrix}, \quad J_{21} = \frac{\partial f_v}{\partial m_x} = -2\mu m_x m_v - 1, \quad J_{22} = \frac{\partial f_v}{\partial m_v} = \mu(1 - m_x^2). \quad (\text{N2})$$

In particular the **(2,1) entry is** $-\frac{h}{2} J_{21} = \frac{h}{2}(2\mu m_x m_v + 1)$.

3.2 The Jacobian in the published scripts

`geometric_integration_computation3.py` (and `4.py`, which is what produced the §6.3 table) builds the Newton matrix as

```
J11,J12=0.0,1.0
J21,J22=(-2.0*mu*m_x*m_v-1.0),(mu*(1-m_x*m_x))
a=1.0-0.5*h*J11; b=-0.5*h*J12; c=0.5*h*J21; d=1.0-0.5*h*J22  # <-- c = +h/2*J21
det=a*d-b*c
xg= ( d*G1 - b*G2)/det
vg= (-c*G1 + a*G2)/det
```

The update lines are the correct Cramer's-rule solution of $DG \delta = G$, but $c = +\frac{h}{2} J_{21}$ **has the opposite sign of the exact (2,1) entry** $-\frac{h}{2} J_{21}$ **in (N2)**. The iteration is therefore Newton's method for a *different* matrix DG^{buggy} that differs from DG only in the sign of the (2,1) entry.

How the sign got there (bug history, verbatim from the scripts). `geometric_integration_computation2` had the *correct* $c = -\frac{h}{2}J_{21}$ but a *wrong* v -update line, `vg -= (c*G1 - a*G2)/det` (an extra sign flip relative to Cramer's rule). The header of `computation3.py` announces a "Bug fix: Newton 2x2 update sign" and fixes the v -update line to the correct `vg -= (-c*G1 ↪ + a*G2)/det` — but in the same edit it *also* flipped c to $+\frac{h}{2}J_{21}$. Fixing one of the two sign errors while introducing the other into the other line, the effective Jacobian is still wrong: the "fix" moved the bug from the update line into the Jacobian entry. Both scripts therefore implement a wrong Newton iteration (in different but equally invalid forms); the one that produced the §6.3 numbers is the wrong-sign-Jacobian version.

3.3 Verification of the corrected step

With $c = -\frac{h}{2}J_{21}$ (exact Jacobian), the Newton step agrees with an **independent** solve of (N1) by `scipy.optimize.root` (hybr, no hand-coded Jacobian) at the machine-precision level, at several states including stiff ones (script output (2)):

state (x, v)	h	correct New- ton	independent root	agreement
(2, 0)	0.01	2 iters, residual 8×10^{-17}	<i>residual</i> 2×10^{-17}	3×10^{-17}
(1.5, -30)	0.01	5 iters, residual 0	residual 0	0
(0.3, -95)	0.01	6 iters, residual 2×10^{-16}	<i>residual</i> 2×10^{-16}	0
(2, 0)	0.5	3 iters, residual 2×10^{-16}	<i>residual</i> 6×10^{-17}	2×10^{-18}
(1.5, -20)	0.5	199 iters, resid- ual 3×10^{-14} (no conver- gence)	residual 1.6×10^{-1} (no conver- gence)	—

(The last row is a genuinely hard state — the jump region at a coarse step — where *both* solvers struggle; it is excluded from the agreement statement. It foreshadows §4: the jump region is where the broken iteration goes catastrophically wrong.)

4. The mechanism of the spurious blow-up

Running the **published** (wrong-sign) and the **correct** Newton from (2, 0) side by side, on the $\mu = 100$ cycle (script outputs (3)–(4), `suspicion63_dive2.py`):

Phase 1 — the slow branch: the two orbits are identical. On the slow drift (the first ~ 81 time units, before the first fast jump), the stage problem is close to linear and both Newton iterations land on the same root: the maximum deviation over the first 120 steps is 1.0×10^{-14} ($h = 0.5$), 7.8×10^{-15} ($h = 0.1$), 8.4×10^{-16} ($h = 0.01$) — machine precision. The published code is therefore *harmless* on the slow branch; nothing about the orbit up to $t \approx 81$ is wrong.

Phase 2 — the first fast jump: the broken Newton diverges in a single step. The first step at which the two orbits differ by more than 1 is, in all three cases, the step that resolves the *first fast jump* of the true orbit:

h	step (t)	state before step	correct Newton	published Newton
0.5	162($t = 81.000$)	$x = 1.0511, v = -0.0850$	199 iters, residual 1.7×10^{-1} , no convergence , lands at $(1.146, 0.466)$ — bounded, mediocre	100iters, residual 3.4×10^{90} , landed $(-1.03 \times 10^{92}, -4.12 \times 10^{92})$
0.1	811($t = 81.100$)	$x = 0.9615, v = -0.4762$	199 iters, residual 4.3×10^{-14} , no convergence , lands at $(-2.753, -73.81)$ — bounded	100iters, residual 9.3×10^{92} , landed $(1.44 \times 10^{93}, 2.88 \times 10^{93})$
0.01	8118($t = 81.180$)	$x = -0.0591, v = -74.16$	6 iters, residual 2.2×10^{-16} , converges, lands at $(-1.147, -143.37)$	100iters, residual 2.1×10^{90} , landed $(-8.72 \times 10^{90}, -1.75 \times 10^{92})$

Two facts about this step. First, the stage system (N1) is **cubic in the endpoint increment** $w = x_g - x$ (verified symbolically, suspicion63_dive2.py output (A)):

$$\frac{\mu}{4}w^3 + \mu x w^2 + (\mu x^2 - \mu + \frac{h}{2} + \frac{2}{h})w + (hx - 2v) = 0, \quad (\text{N3})$$

so at the jump state there are several real roots (e.g. at $h = 0.5$: $w \in \{-4.061, -0.167, +0.021\}$); the *correct* Newton, started at the current state, converges to the nearby root — the true IM step. The wrong-sign Jacobian has a different error-propagation spectrum and, at this state, its iteration **diverges** instead: after the 100-iteration cap the iterate is 10^{27} – 10^{29} away from *any* root (recomputed stage residual 10^{81} – 10^{90}).

Second — the silent-failure layer — the published loop is a bare `for _ in range(100)` with a return *after* the loop and **no convergence check**: when the iteration has not converged, the script returns the last iterate as if it were the step. A single garbage step is then indistinguishable, inside the code, from a legitimate one.

Phase 3 — the garbage is carried along. Once the orbit is at $|x|, |v| \sim 10^{27}$ – 10^{29} , the subsequent “steps” are garbage-in-garbage-out; the run's maximum $\max |x|$ is *exactly* the value produced at the single failed step in all three cases (1.03×10^{27} at $h = 0.5$, 1.44×10^{29} at $h = 0.1$, 8.73×10^{29} at $h = 0.01$ — the re-run reproduces the raw output digit for digit). There is no gradual instability of the implicit midpoint method to be observed: the “blow-up” is instantaneous, it happens at the same physical moment for all three step sizes (the first fast jump, $t \approx 81$), and it is entirely a property of the broken Newton solve.

This also explains the essay's observation that the failure “is not a step-size artifact (it persists at $h = 0.01$)”: the broken iteration fails at the jump regardless of h , because the defect is in the Jacobian, not in the step size.

5. What the implicit midpoint rule *actually* does on $\mu = 100$

Correct Newton (exact Jacobian; cross-validated as in §3.3), start $(2, 0)$, true cycle $T_{\text{ref}} = 162.8371$ (Radau, 10^{-10} tolerance), amplitude 2, $\max |v| \approx 134$:

Three periods (~ 488.5 time units):

method (step h)	period T	$\max x $	$\max v $	status
implicit mid-point, $h = 0.5$	— (no clean crossings)	3.03	15.9	bounded , jump unresolved
implicit mid-point, $h = 0.1$	286.50	2.95	73.8	bounded , distorted (T off by +76%)
implicit mid-point, $h = 0.01$	179.08	2.09	143.9	bounded , distorted (T off by +10%)

Longer runs (boundedness check):

h , periods	T (last 3)	$\max x $	$\max v $	status
0.01 , 20 periods	179.13	2.09	144.2	bounded
0.1 , 10 periods	193.20	4.85	127.2	bounded
0.5 , 5 periods	—	3.05	16.2	bounded

Per-period envelope, correct IM at $h = 0.01$, 10 periods: $\max |x| \in [2.06, 2.09]$, $\max |v| \in [129, 144]$ — stable envelope, no growth, no escape. This is exactly the reviewer's independent observation ("bounded, values up to ~ 100 -150, even partially recovers, over 20 periods").

The correct method therefore **does** exhibit the defect the essay set out to demonstrate — but it manifests as *cycle distortion*, not escape: the fast jump is smeared (the orbit does not snap onto the slow manifold), the period is too long (+10% at $h = 0.01$), and the distortion grows with h (+76% at $h = 0.1$; at $h = 0.5$ the jump is not resolved at all). This is the genuine, *A*-stable-but-not-*L*-stable behavior: with $R(-3\mu h) \rightarrow -1$ as $\mu h \rightarrow \infty$ (values: $R(-3) = -0.2$, $R(-30) = -0.875$, $R(-150) = -0.9737$), the fast mode is attenuated toward *magnitude 1 with a sign flip* rather than to 0, so the jump is mishandled and the cycle is distorted — but the nonlinear orbit remains bounded at these step sizes. Notably, the essay's own mechanistic argument (residual $O(0.2)$ fast component "re-amplified by the unstable central region, so the orbit diverges") **over-predicts**: it predicts escape, while the correctly-implemented orbit stays bounded even at $h = 0.5$ where $R(-150) \approx -0.974$. The re-amplification degrades the cycle; it does not, at these parameters, destroy it.

The rest of the §6.3 table is genuine. The Strang-split rows (no Newton involved) reproduce exactly: $h = 0.5 \Rightarrow \max |x| = 2.01 \times 10^{15}$ (blow-up, as the essay's central-region amplification argument correctly predicts for the exact-damping split), $h = 0.2 \Rightarrow T = 151.40$, $\max |x| = 4.59 \times 10^8$ (wrong period, as printed in the raw output — note the essay's " $\sim 4.6 \times 10^5$ " is the misquotation flagged in §2). The Radau IIA rows reproduce exactly: $T = 162.8371$ at both $h = 0.5$ and $h = 1.0$, identical to the reference to the displayed digits.

6. Consequences for the essay

What survives. The essay's structural argument is intact and is, in fact, *strengthened* in one respect: the correct implicit midpoint rule really does fail to handle the stiff jump (bounded but distorted, with the distortion growing with h), while the *L*-stable Radau IIA is exact to the displayed digits at $h = 1.0 \gg 1/\mu$. The tension "symplecticity vs. *L*-stability" in §6.5, the stability-function facts of §2.3, and the Strang-split analysis of §6.3 all stand on verified ground. The qualitative point — "large uncontrolled error on the fast jump" — is already enough for the essay's thesis, exactly as the review anticipated.

What must change.

1. **Withdraw the "escapes to $\sim 10^{30}$ " claim** (and the "blow-up" status of the three implicit-midpoint rows). Replace with the verified bounded-but-distorted behavior:

$T = 179.08$ (+10%) at $h = 0.01$ with $\max |v| \approx 144$; $T = 286.5$ (+76%) at $h = 0.1$; jump unresolved at $h = 0.5$. The corrected statement is *stronger in honesty* and still sufficient: the method that is symplectic and A -stable produces a materially wrong cycle in the stiff regime, and the method that is right is not symplectic.

2. **Fix the table to the raw values** (and fix the Strang $h = 0.2$ row, $4.6 \times 10^5 \rightarrow 4.6 \times 10^8$). As printed, the table disagrees with its own raw output by factors of 10^3 in two rows.
3. **Softening of the mechanism paragraph.** The " $R(-3) = -0.2$ residual re-amplified to divergence" story should be replaced by the distortion data: the non- L -stable damping ($R \rightarrow -1$) smears the jump and biases the period, with the bias growing as h grows; it does not, at these parameters, drive the orbit to infinity.
4. **The $\mu = 1$ (non-stiff) IM row** of §6.3 is unaffected and remains valid (there the Newton converges at every step; $T = 6.663333$, amplitude 2.0086, as in results3.txt).

Recommendations for the code (so the defect class cannot recur silently):

- Correct the Jacobian entry to $c = -\frac{h}{2} J_{21}$ (N2).
- **Never return an unconverged Newton iterate silently:** after the iteration cap, check the residual and raise (or fall back to a guaranteed-convergent solver, e.g. a damped Newton with backtracking or a direct root finder). This single check would have turned the "blow-up to 10^{30} " into a loud, local, diagnosable failure at $t \approx 81$ instead of a silent global one.
- Re-run §6.3 and regenerate the table from the raw output without transcription.

7. Reproducibility

All computations in this analysis were run with the project venv (D:/Source/hermes-[dir](#)/.venv, Python 3.11.9, NumPy 2.4.6, SciPy 1.17.1, SymPy 1.14.0, matplotlib 3.11.2):

file	content
'suspicion63_analysis.py' → 'suspicion63_results.txt'	(1) symbolic Jacobian of the stage equations + harmonic-oscillator symplectic check; (2) correct-Newton vs. independent-root cross-checks; (3) convergence diagnostics of the published vs. correct Newton; (4) buggy vs. correct orbits, 3 periods and longer, per-period spikes; (5) Strang reproduction; (6) Radau rows; (7) stability-function values; (8) harmonic sanity
'suspicion63_dive2.py' → 'suspicion63_dive2_results.txt'	symbolic verification of the stage cubic (N3); first-divergence localization; buggy-iterate residuals and distance to the true stage roots at the jump

'suspicion63_figure.png'	$h = 0.01$ ', $\mu = 100$: published Newton (red, $ x $ log scale — jumps to 10^{30} at $t \approx 81$) vs. correct Newton (blue, bounded relaxation cycle)
--------------------------	--

Run any of them with:

```
D:/Source/hermes-dir/.venv/Scripts/python.exe suspicion63_analysis.py
```

The buggy-Newton reproduction matches `geometric_integration_results4.txt` digit for digit on all three implicit-midpoint rows, confirming that the published table's (raw) numbers are fully explained by the two defects identified in §3–§4, with no residual mystery.

This analysis modifies nothing in the v1 essay; it is a companion document. The v1 essay's §6.3, §6.5 and §9 references to the " $\sim 10^{30}$ " blow-up are the only passages affected, and the correction to §6.3 is a replacement of one table and one paragraph, not a structural change.